西南交通大学
Southwest Jiaotong University

SCAI011112 视觉计算

# Transformer 网络

计算机科学与技术专业

2025-2026 (2)

# 本章主要内容

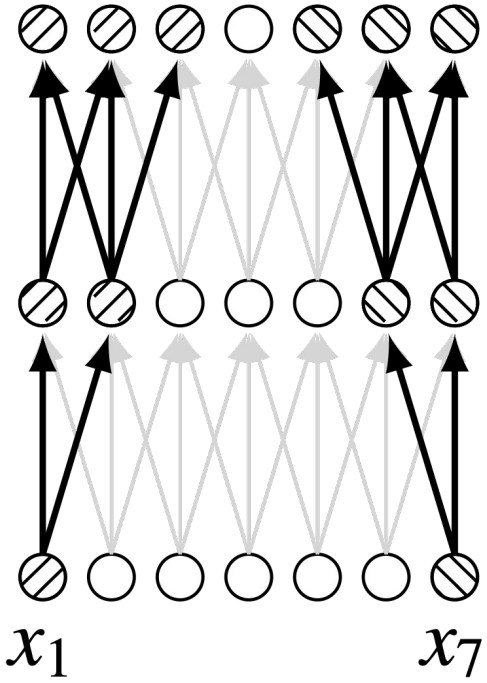
- 卷积网络的局限
- 注意力机制
- 词元
- 注意力层
- 完整Transformer 架构
- 置换等变性
- 位置编码
- 掩码注意力
- 比较 CNN 与Transformer

# 引言

- Transformer 是一种新兴的架构，它对CNN的思想进行了推广和扩展。最初应用于语言建模，后来被用于视觉中。
- 与CNN类似，Transformer 将信号处理问题分解为涉及“独立且相同处理”块（chunks）的阶段。然而，它们还包含了一些能够跨块混合信息的层，称为注意力层，从而使整个流程能够建模块之间的依赖关系。

# CNN 的局限

- CNN的**局部性思想**：图像的不同局部区域可以被安全地独立处理。
- **局限性**：不擅长处理信息的全局信息。



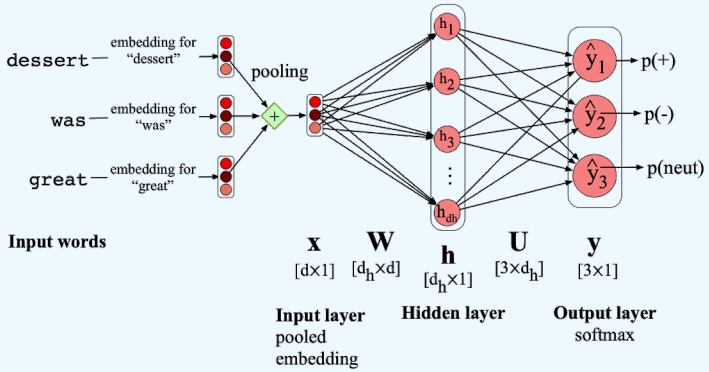

卷积核大小为3的2层CNN，任务是比较$x_1$和$x_7$。它无法做到这一点：没有任何神经元同时连接到$x_1$和$x_7$

CNN 要处理全局信息的方式：

- 方式 1：增加滤波器的卷积核大小
- 方式 2：通过堆叠层数来增加深层神经元的感受野

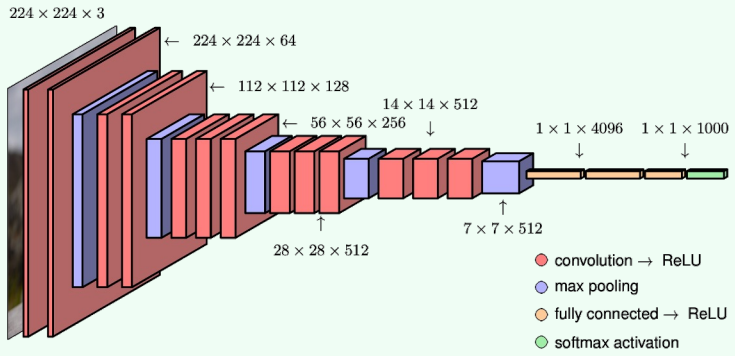
# MLP, CNN, and RNN

## 全连接网络



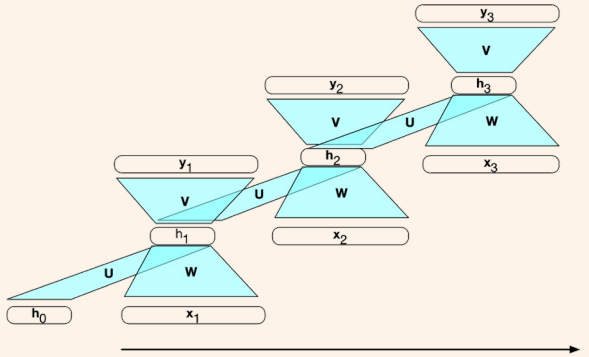

- 可以全局信息传递
- 参数量大
- 无法处理变长数据
- 不具备平移、旋转不变性

## 卷积网络




- 参数量小（权值共享）
- 具备平移不变性
- 并行化计算
- 长程依赖建模较弱（需堆叠）

## 循环网络




- 处理任意长度序列
- 具备“时序记忆”能力
- 并行化困难
- 长程依赖建模能力弱

# 注意力机制







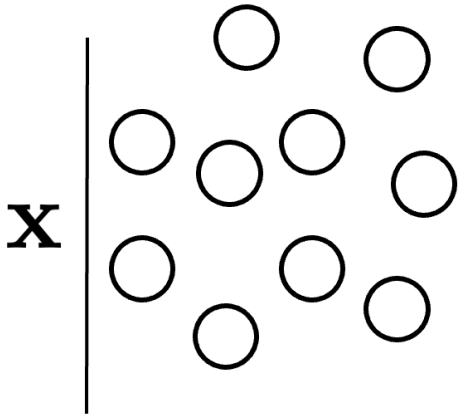
所有黑色牌的数字之和为多少?

# 注意力机制

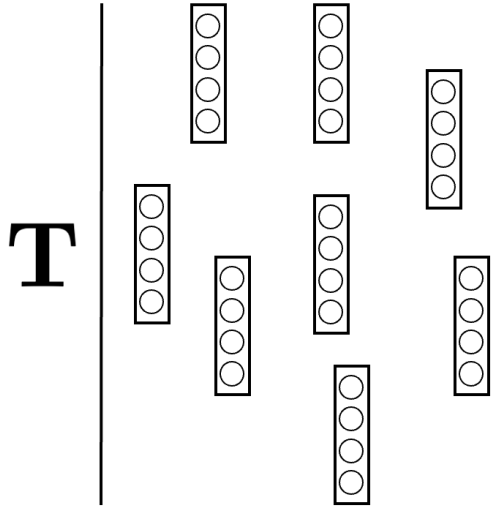
- 注意力是一种高效处理全局信息的策略，它只聚焦于信号中对当前任务最显著的部分，关注那些突出的元素，而不是一次性接收整个场景的所有信息
- 在神经网络中，注意力遵循同样的直观思路。第 $l$ 层的一组神经元可能会关注第 $l-1$ 层的一组神经元，以决定它们应该做出怎样的响应
  - 如果我们“要求”这组神经元报告输入图像中任何汽车的颜色，那么它们就应该将注意力导向上一层中代表汽车颜色的神经元

# 词元（Token）

- 词元是神经元的另一种的组合形式，可以将其视为封装的信息组，本章考虑的单一的词元通过一个列向量 $\mathrm{t} \in \mathbb{R}^{\mathrm{d}\times 1}$来表示，有时被称为代码向量



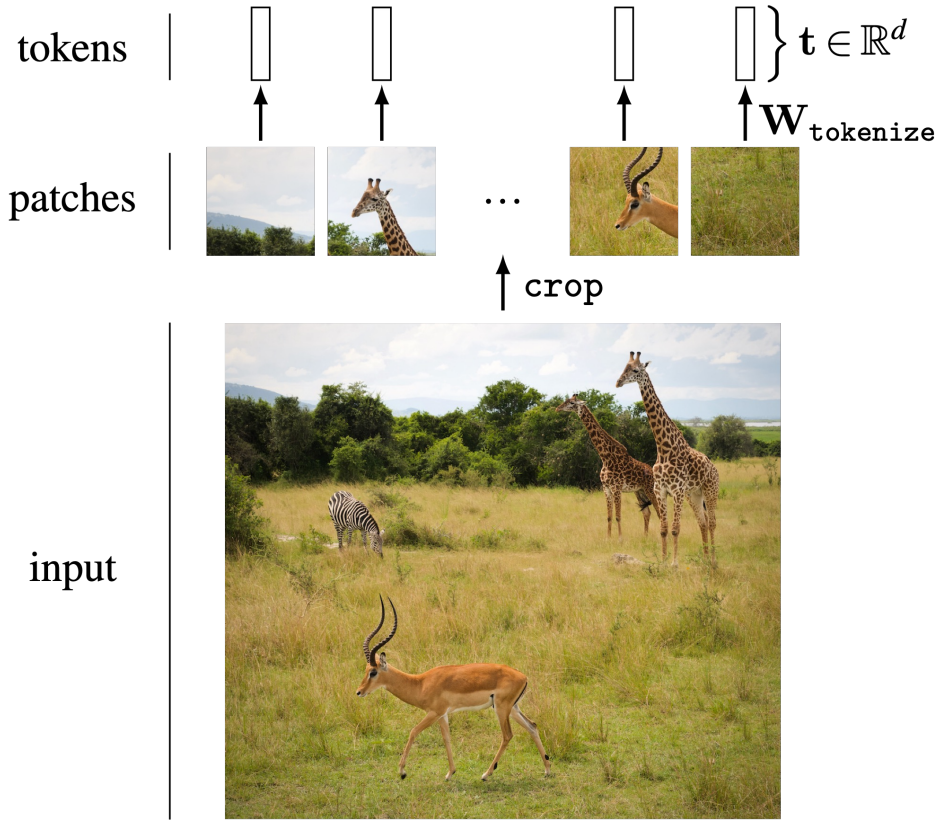

神经元集合




词元集合

# 数据词元化

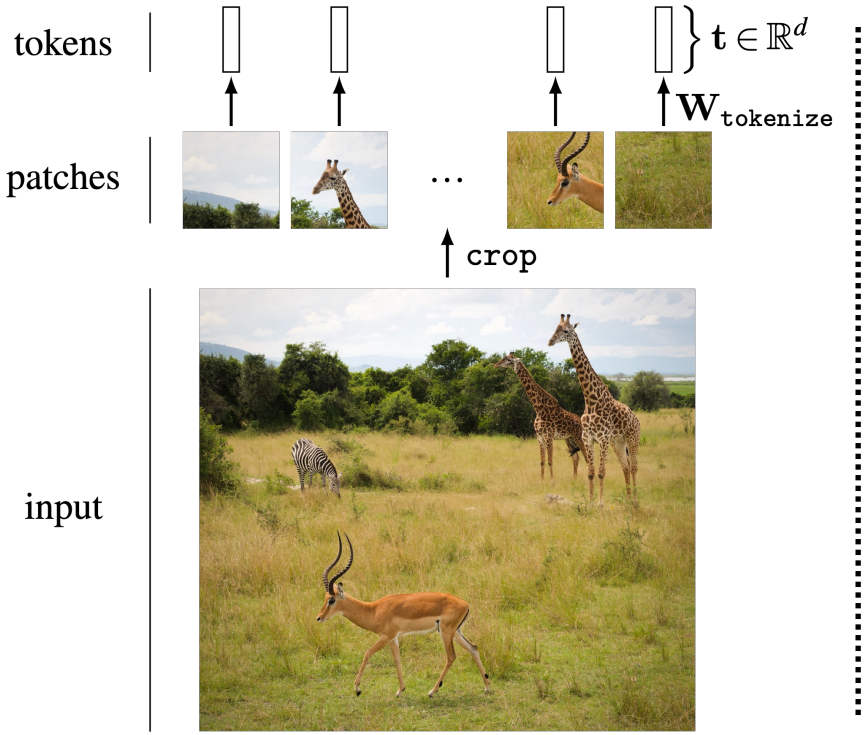
- 对图像进行词元化，用一个 Token 来代表图像中的每一个图像块

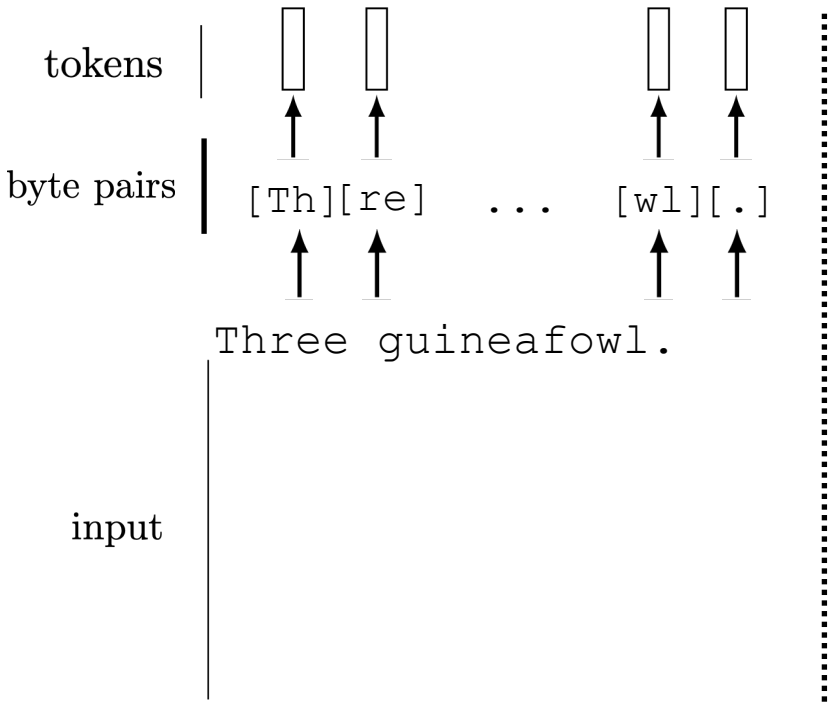


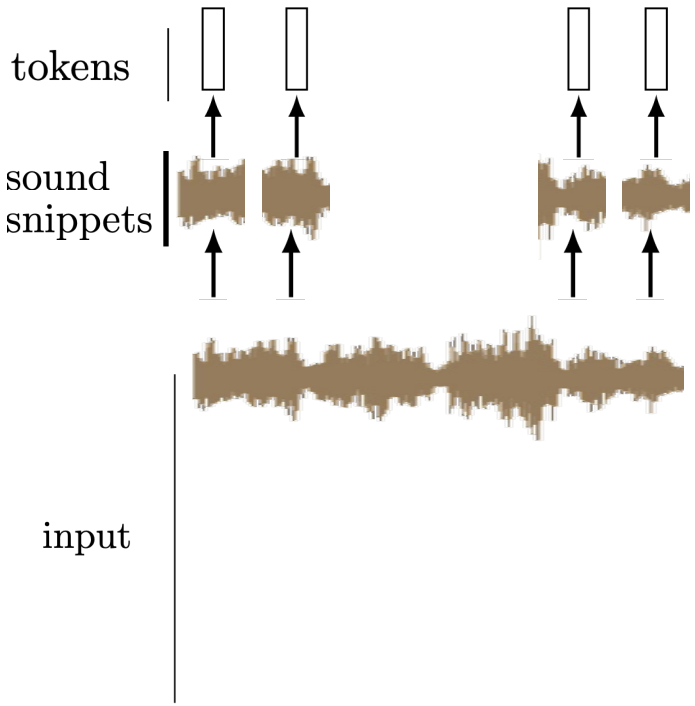

- 当每个图像块都由一个词元表示时，整幅图像就对应于一个词元阵列。
- **直接向量化：**不涉及任何学习过程，只是数据的重排
  - 16*16像素，RGB三通道，拉长成768维（256+256+256）向量 x
- **低维投影：**引入可学习的嵌入矩阵$\mathrm{W}_{\text{tokenize}}$，$\mathrm{z} = \mathrm{W}_{\text{tokenize}}\mathrm{x}$

# 数据词元化

You can tokenize anything.
General strategy: chop the input up into chunks, project each chunk to a vector.

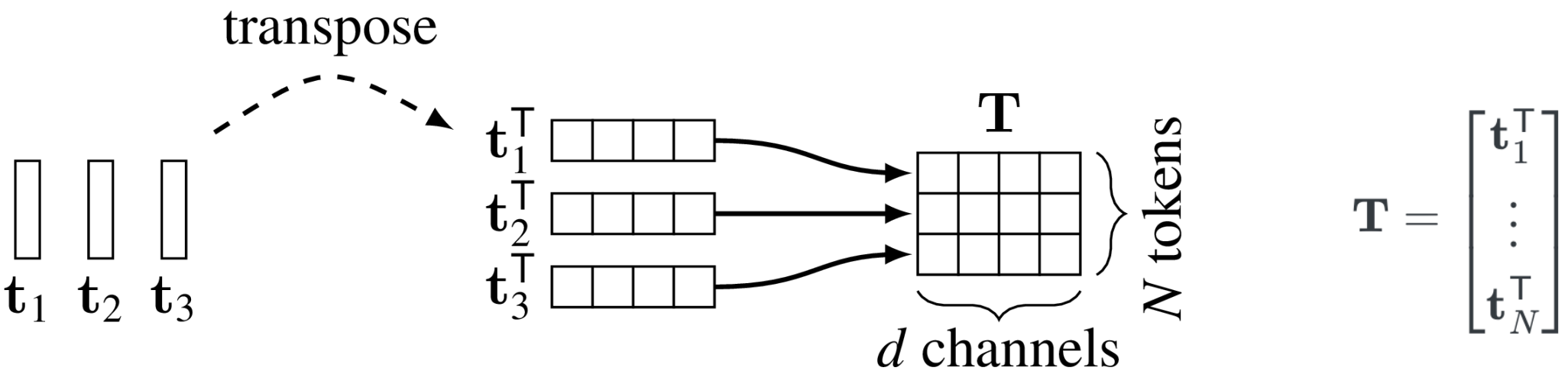


# 词元序列

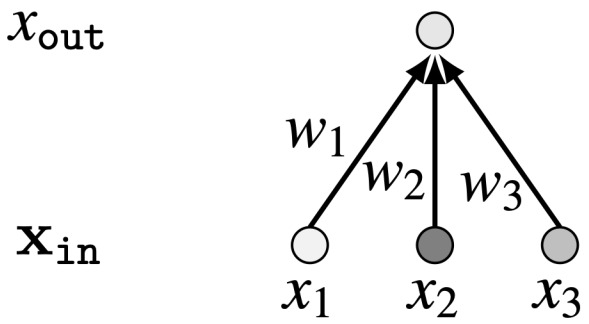
- 神经网络层运行在神经元阵列之上
  - MLP的输入是一个列向量 $\mathbf{x}$，其每一行都是一个标量神经元。
- Transformer 运行在词元阵列之上
  - 矩阵 $\mathbf{X}$ 仅仅是向量值词元的一维阵列的一种便捷表示方式。




- Transformer针对词元两项核心操作：**线性组合**，**逐点非线性变换**

# 词元的线性组合

## Linear combination of neurons

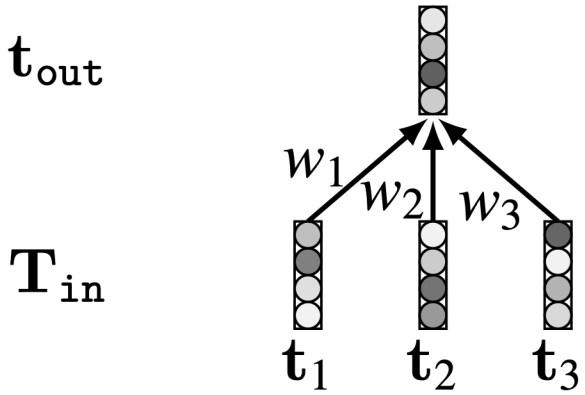



$$x_{out} = w_1 x_1 + w_2 x_2 + w_3 x_3$$

$$x_{out}[i] = \sum_{j=1}^{N} w_{ij} x_{in}[j]$$

$$\mathbf{x}_{out} = \mathbf{W}\mathbf{x}_{in}$$

## Linear combination of tokens




$$\mathbf{t}_{out} = w_1 \mathbf{t}_1 + w_2 \mathbf{t}_2 + w_3 \mathbf{t}_3$$

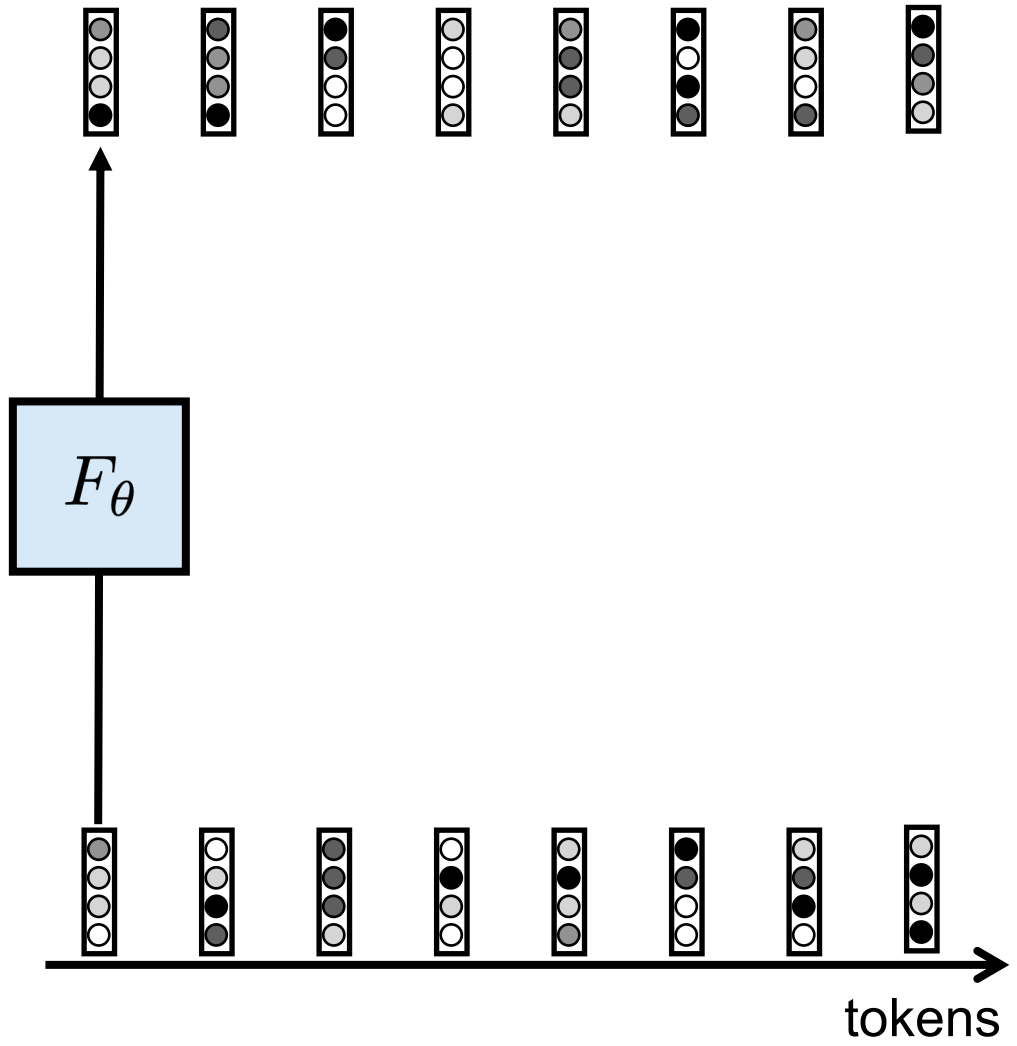
$$\mathbf{T}_{out}[i,:] = \sum_{j=1}^{N} w_{ij} \mathbf{T}_{in}[j,:]$$

$$\mathbf{T}_{out} = \mathbf{W}\mathbf{T}_{in}$$

# 逐点非线性变换

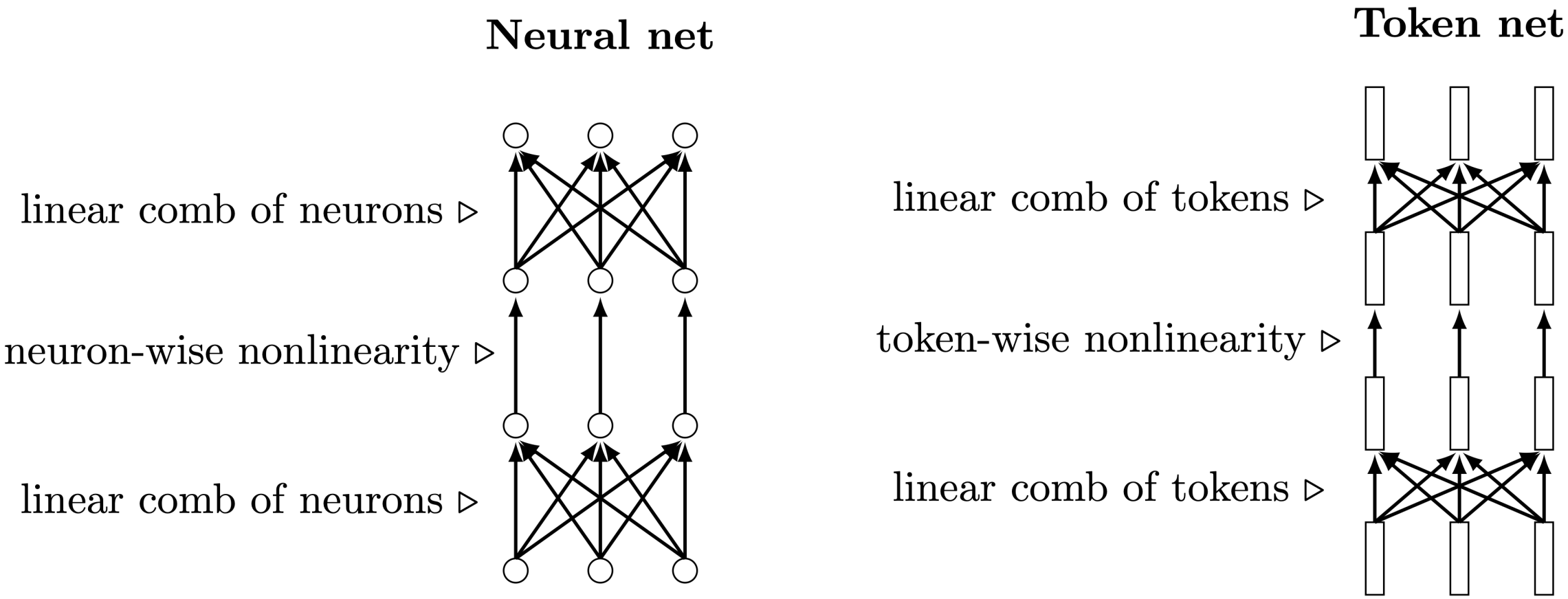
$$\mathbf{x}_{\text{out}} = \begin{bmatrix} \texttt{relu}(x_{\text{in}}[0]) \\ \vdots \\ \texttt{relu}(x_{\text{in}}[N-1]) \end{bmatrix}$$

$$\mathbf{T}_{\text{out}} = \begin{bmatrix} F_\theta(\mathbf{T}_{\text{in}}[0,:]) \\ \vdots \\ F_\theta(\mathbf{T}_{\text{in}}[N-1,:]) \end{bmatrix}$$



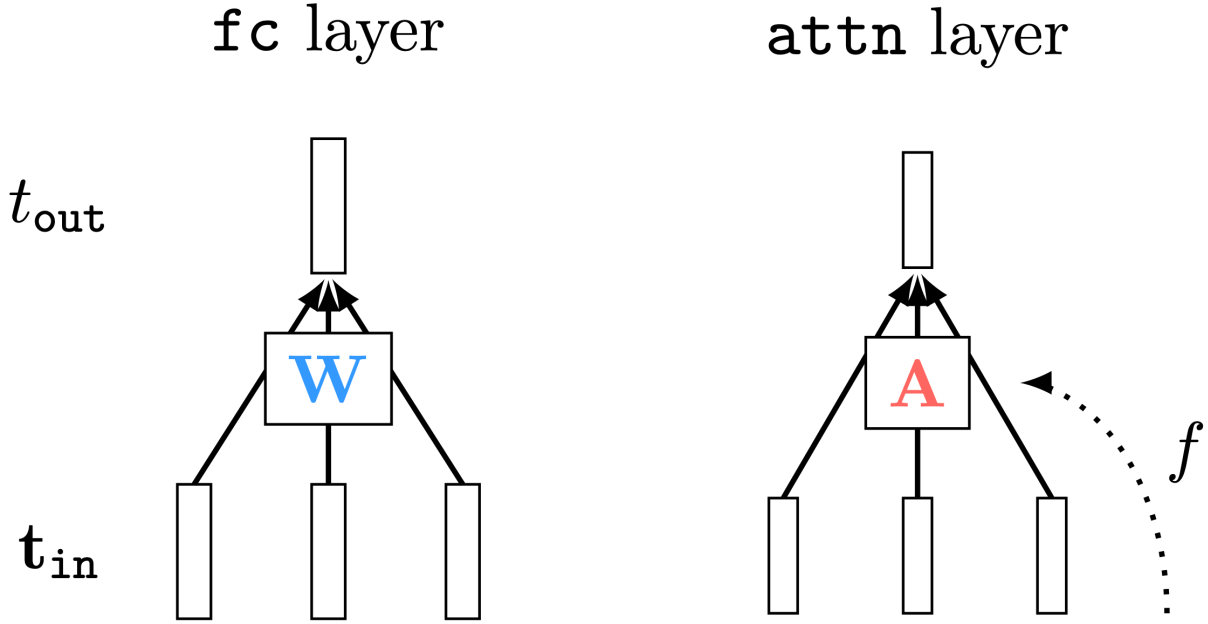

主流选择将设为 MLP，具有可学习参数，使得逐点运算更具表达力且参数更丰富。

# 词元网络（Token Net）



# 注意力层

- 注意力层采用特殊的 tokens线性组合方式，它是一种具有**数据依赖权重**的 FC 层
- 或者叫**动态池化**，即采用加权平均的池化，其中权重根据输入数据动态确定




W 为自由参数

$$\mathbf{A} = f(\ldots) \qquad \triangleleft \text{ attention}$$

$$\mathbf{T}_{\text{out}} = \mathbf{A}\mathbf{T}_{\text{in}}$$

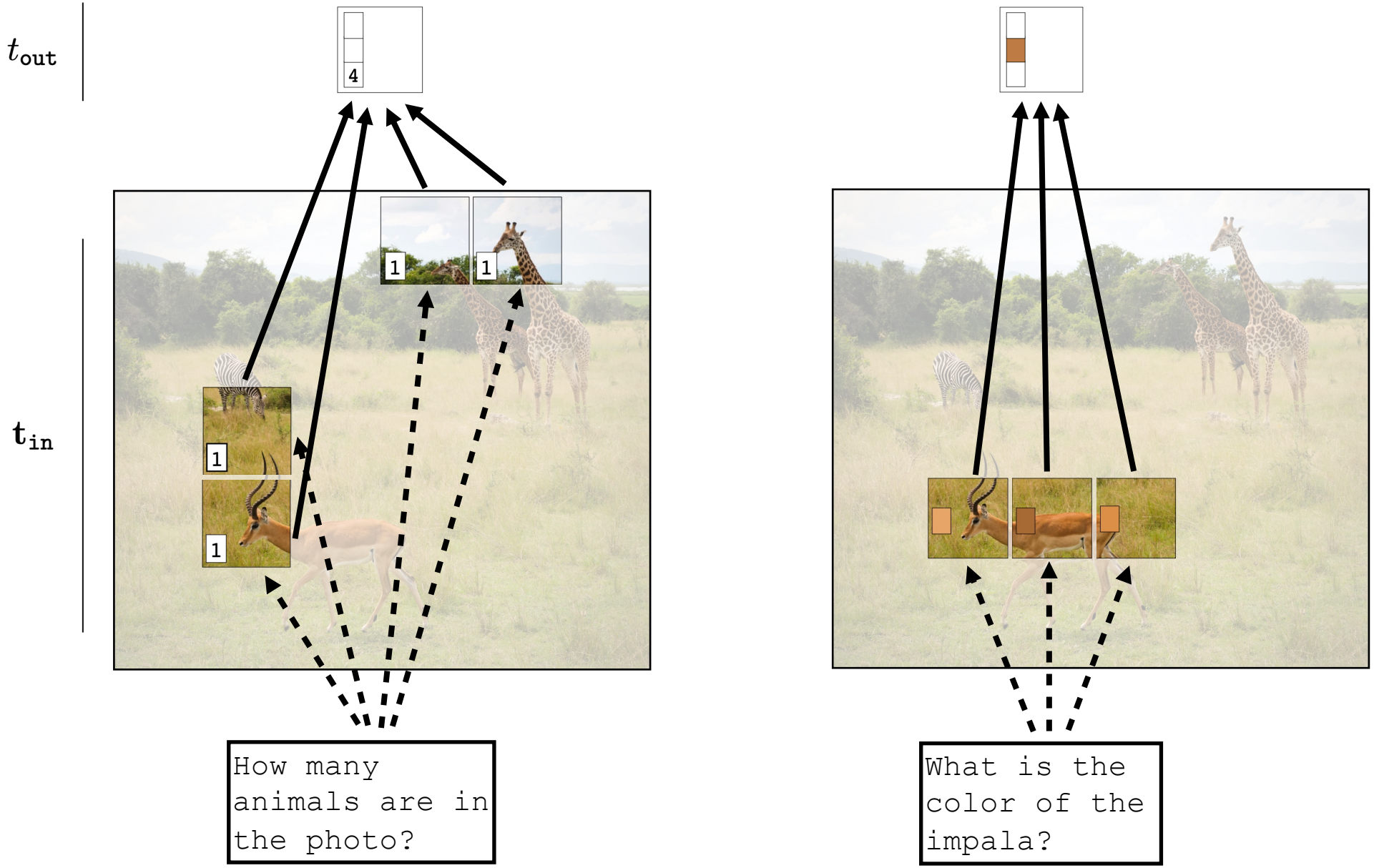
- A 是某些输入数据的函数
- 这些数据告诉我们应该关注（即在加权求和中分配较高的权重）哪些词元
- A 通常仅包含非负值
- f 实际上是确定词元的组合权重

$t_{out}$

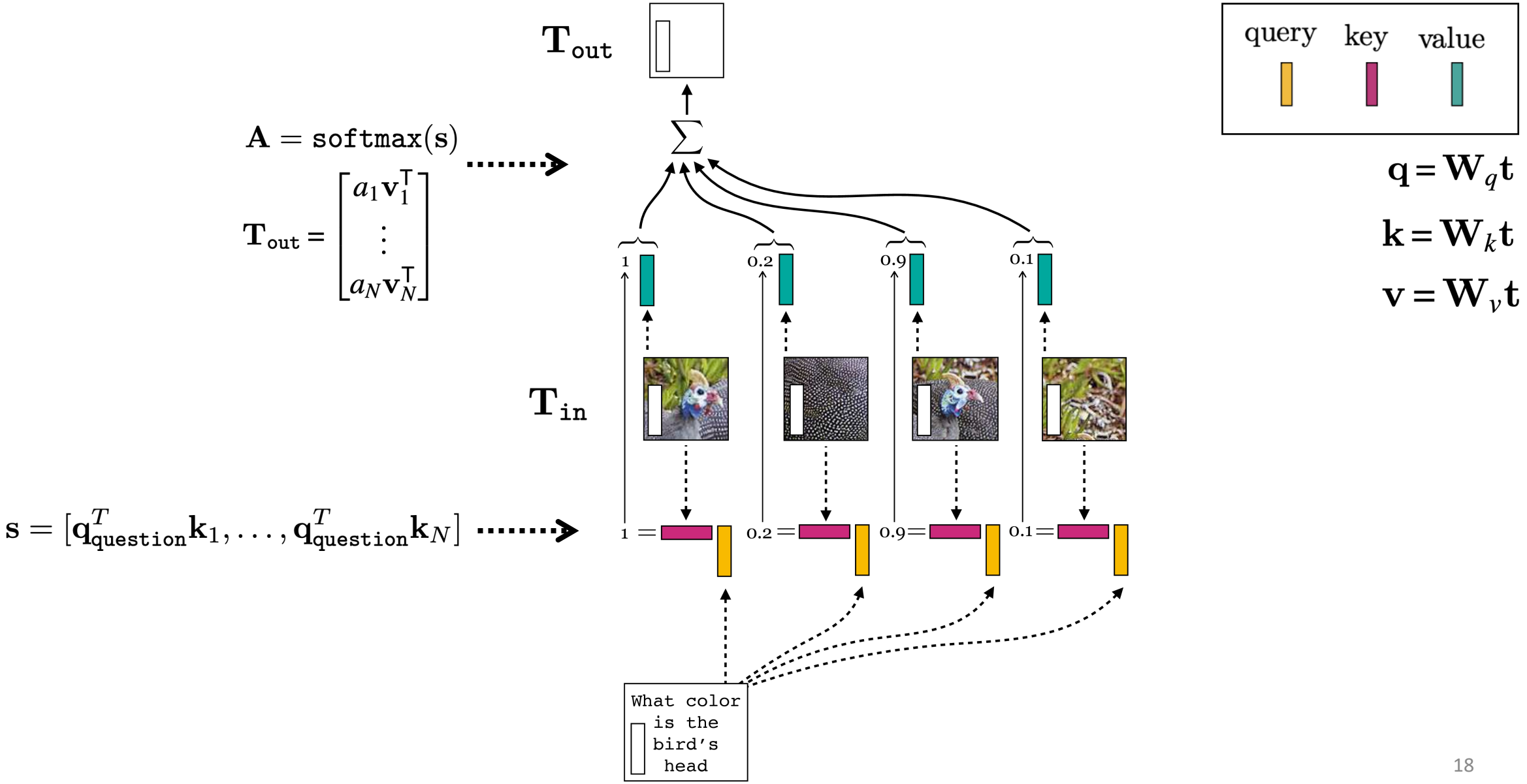
$t_{in}$



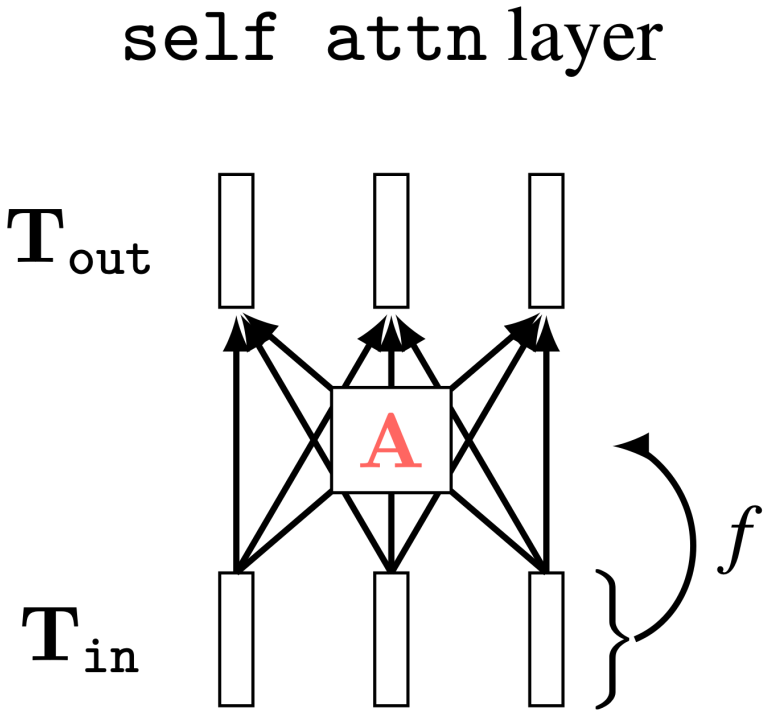
How many animals are in the photo?


$t_{out}$
$t_{in}$
4
1
1
1
1
How many animals are in the photo?
What is the color of the impala?

# Query-Key-Value 注意力



# 自注意力（Self-Attention）




$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{q}_1^{\mathsf{T}} \\ \vdots \\ \mathbf{q}_N^{\mathsf{T}} \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^{\mathsf{T}} \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^{\mathsf{T}} \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_q^{\mathsf{T}} \quad \triangleleft \quad \text{query matrix}$$

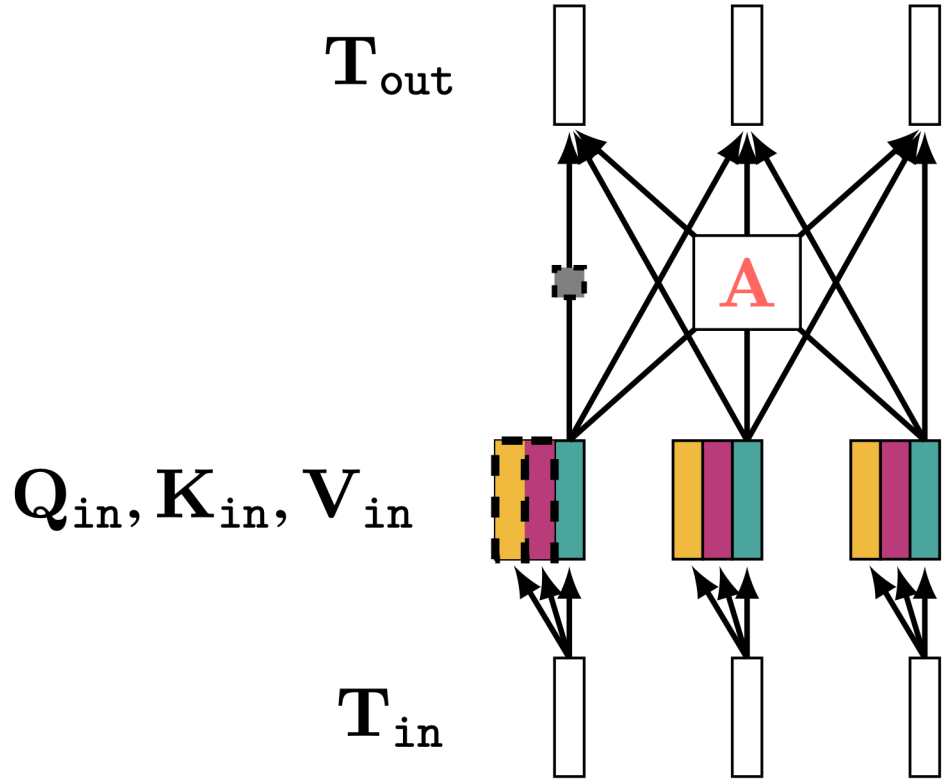
$$\mathbf{K}_{\text{in}} = \begin{bmatrix} \mathbf{k}_1^{\mathsf{T}} \\ \vdots \\ \mathbf{k}_N^{\mathsf{T}} \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^{\mathsf{T}} \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_N)^{\mathsf{T}} \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_k^{\mathsf{T}} \quad \triangleleft \quad \text{key matrix}$$

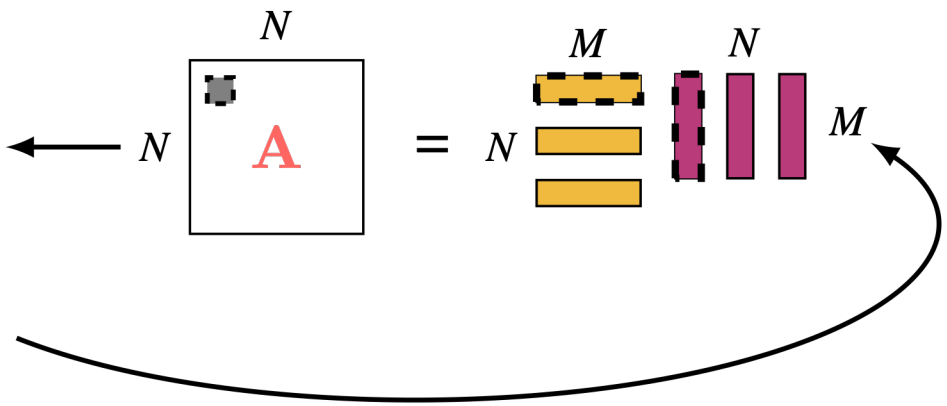
$$\mathbf{V}_{\text{in}} = \begin{bmatrix} \mathbf{v}_1^{\mathsf{T}} \\ \vdots \\ \mathbf{v}_N^{\mathsf{T}} \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^{\mathsf{T}} \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^{\mathsf{T}} \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_v^{\mathsf{T}} \quad \triangleleft \quad \text{value matrix}$$

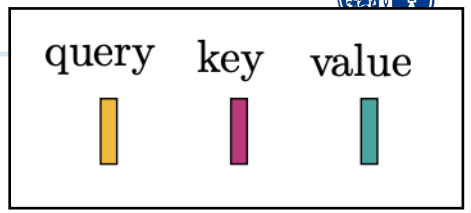
$$\mathbf{A} = f(\mathbf{T}_{\text{in}}) = \texttt{softmax}\left(\frac{\mathbf{Q}_{\text{in}} \mathbf{K}_{\text{in}}^{\mathsf{T}}}{\sqrt{m}}\right) \quad \triangleleft \quad \text{attention matrix}$$

$$\mathbf{T}_{\text{out}} = \mathbf{A}\mathbf{V}_{\text{in}}$$

# 自注意力层




$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{q}_1^{\mathsf{T}} \\ \vdots \\ \mathbf{q}_N^{\mathsf{T}} \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^{\mathsf{T}} \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^{\mathsf{T}} \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_q^{\mathsf{T}} \quad \triangleleft \quad \text{query matrix}$$

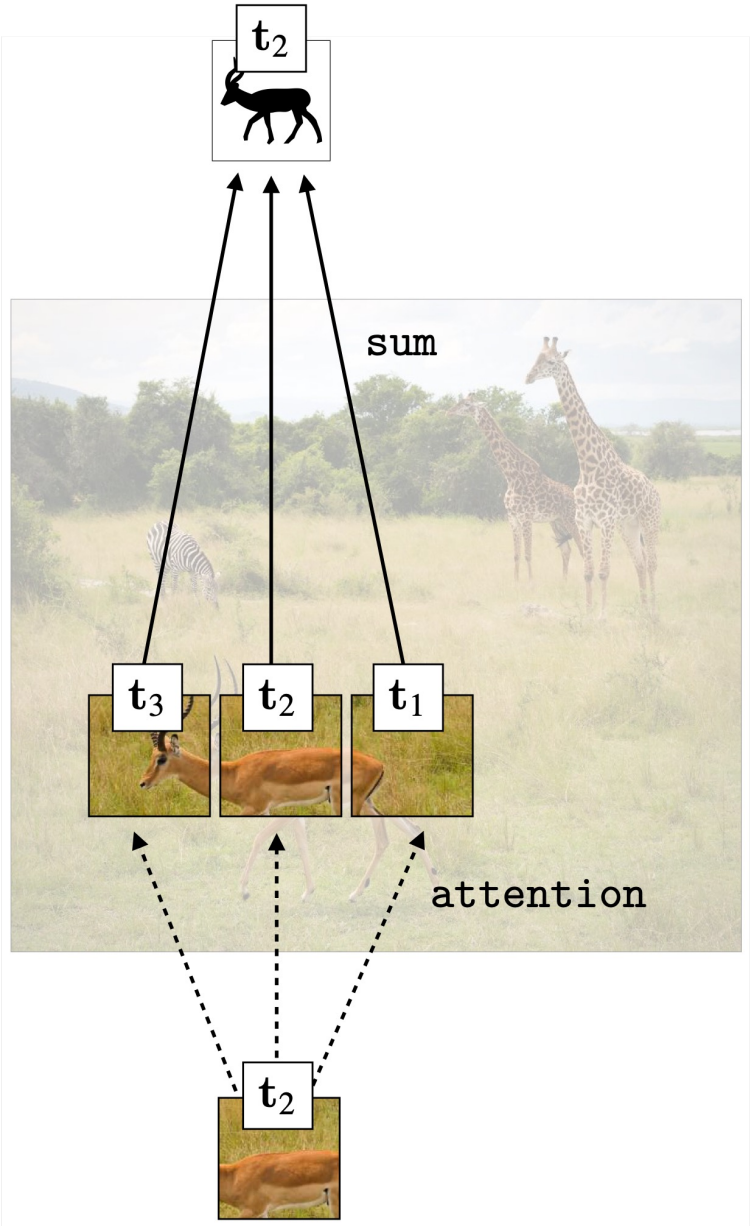
$$\mathbf{K}_{\text{in}} = \begin{bmatrix} \mathbf{k}_1^{\mathsf{T}} \\ \vdots \\ \mathbf{k}_N^{\mathsf{T}} \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^{\mathsf{T}} \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_N)^{\mathsf{T}} \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_k^{\mathsf{T}} \quad \triangleleft \quad \text{key matrix}$$

$$\mathbf{V}_{\text{in}} = \begin{bmatrix} \mathbf{v}_1^{\mathsf{T}} \\ \vdots \\ \mathbf{v}_N^{\mathsf{T}} \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^{\mathsf{T}} \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^{\mathsf{T}} \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_v^{\mathsf{T}} \quad \triangleleft \quad \text{value matrix}$$

$$\mathbf{A} = f(\mathbf{T}_{\text{in}}) = \texttt{softmax}\left(\frac{\mathbf{Q}_{\text{in}} \mathbf{K}_{\text{in}}^{\mathsf{T}}}{\sqrt{m}}\right) \quad \triangleleft \quad \text{attention matrix}$$

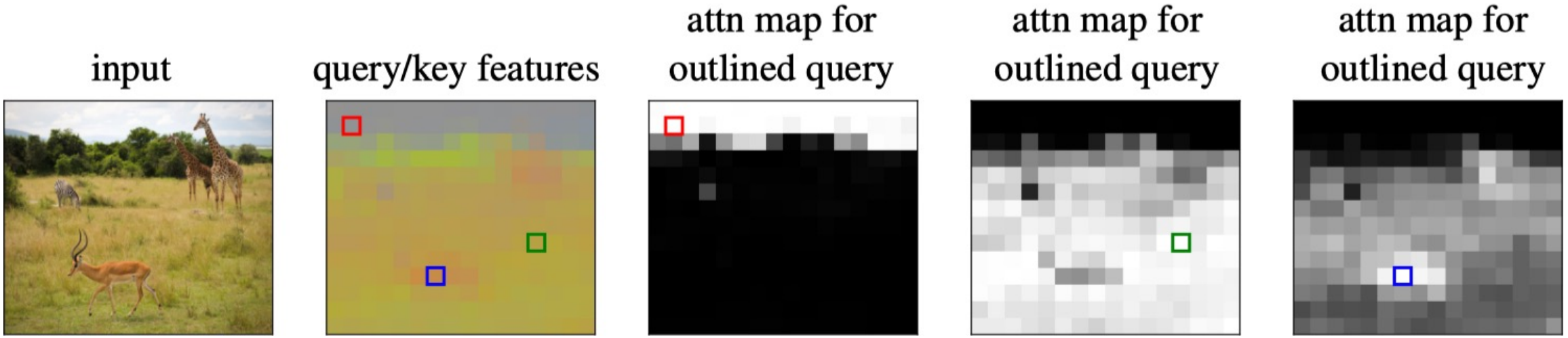
$$\mathbf{T}_{\text{out}} = \mathbf{A}\mathbf{V}_{\text{in}}$$

# 自注意力层




- t2 关注代表黑斑羚其他部位的词元，并利用它们将 t2 优化为一个**更抽象的词元向量**，从而捕捉到“黑斑羚”这一标签
- 在给定周围其他相关图像块的**上下文信息**时，识别某个特定的图像块会变得更容易
- 减少了三个被关注的黑斑羚图像块之间不共享的噪声，从而**放大了它们之间的共性** — 即“黑斑羚”这一标签

# 自注意力层




**矩阵 A 是输入数据的函数**：

- **动态性：** Query 变了，生成的感受野（高亮区域）就完全变了。这在 CNN 中是不可想象的，CNN 的卷积核感受野是固定形状的
- **打破空间限制：** 注意力机制完全无视了空间距离。红框位于左上角，但它依然能给予右上角的天空极高的权重。只要特征匹配，“天涯若比邻”

# 多头自注意力（Multi-head Self-Attention）

- Query/key/value 映射矩阵 $W_q$，$W_k$，$W_v$ 定义了用于将查询与键进行匹配的相似度概念
- 一个单一的自注意力层只能以一种方式来衡量相似度
- Transformer 使用多头自注意力，并行地运行 k 个注意力层

$$
\begin{aligned}
s_{ij} &= \mathbf{q}_i^\top \mathbf{k}_j \\
&= (\mathbf{W}_q \mathbf{t}_i)^\top \mathbf{W}_k \mathbf{t}_j \\
&= \mathbf{t}_i^\top \mathbf{W}_q^\top \mathbf{W}_k \mathbf{t}_j \\
&= \mathbf{t}_i^\top \mathbf{S} \mathbf{t}_j
\end{aligned}
$$

$$
\mathbf{S} = \mathbf{W}_q^\top \mathbf{W}_k
$$

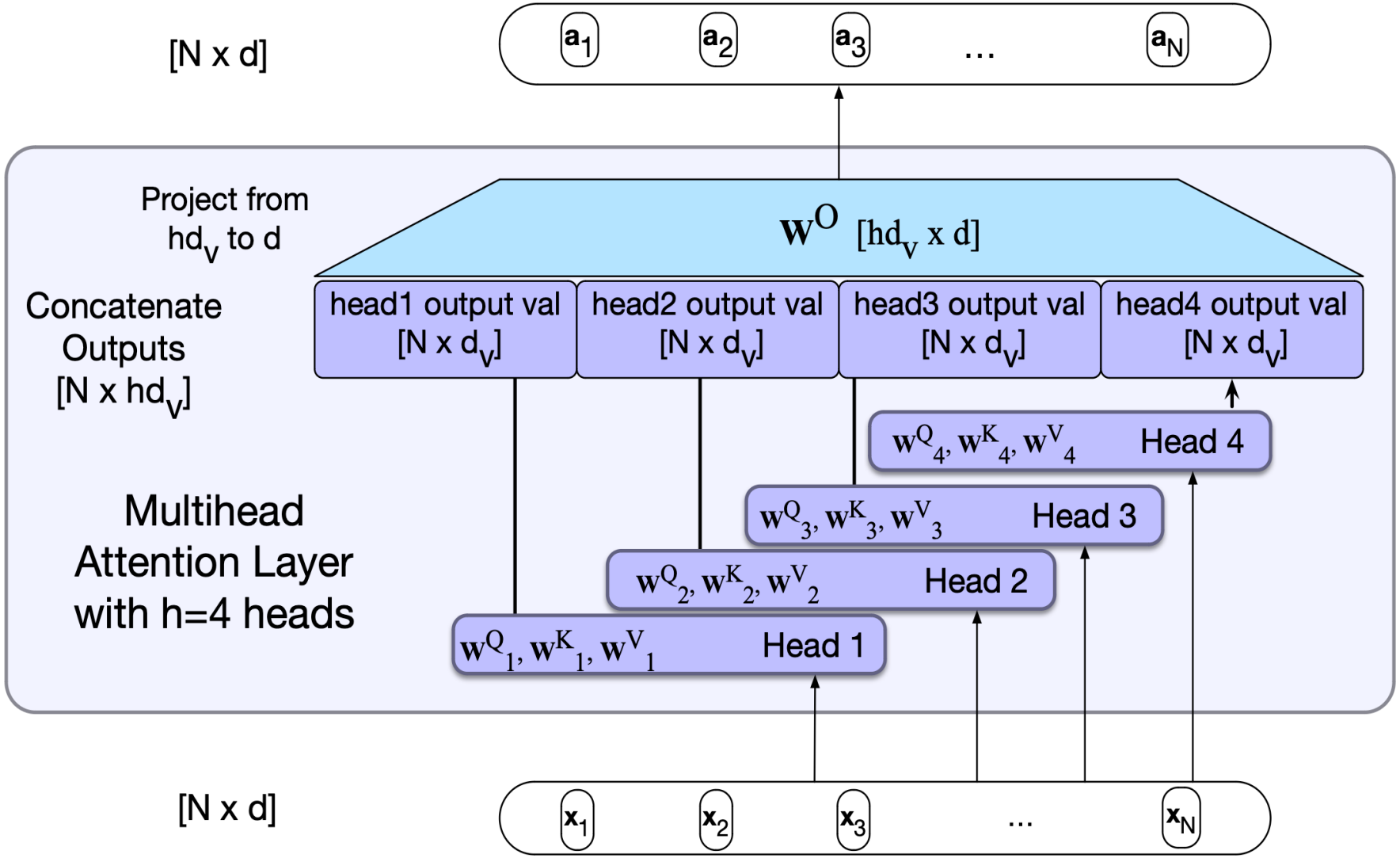
$$
\mathbf{T}_{\text{out}}^i = \texttt{attn}^i(\mathbf{T}_{\text{in}}) \quad \text{for } i \in \{1, \ldots, k\}
$$

$$
\bar{\mathbf{T}}_{\text{out}} = \begin{bmatrix} \mathbf{T}_{\text{out}}^1[0,:] & \cdots & \mathbf{T}_{\text{out}}^k[0,:] \\ \vdots & \vdots & \vdots \\ \mathbf{T}_{\text{out}}^1[N-1,:] & \cdots & \mathbf{T}_{\text{out}}^k[N-1,:] \end{bmatrix}
$$

$$
\mathbf{T}_{\text{out}} = \bar{\mathbf{T}}_{\text{out}} \mathbf{W}_{\texttt{MSA}}
$$

$$
\bar{\mathbf{T}}_{\text{out}} \in \mathbb{R}^{N \times kv} \qquad \mathbf{W}_{\texttt{MSA}} \in \mathbb{R}^{kv \times d}
$$

# 多头自注意力（Multi-head Self-Attention）



# Transformer Blocks

- **Feedforward layer**
  - *N* position-wise networks, one at each position
  - Networks are independent for each position
  - The weights are the same for each position, but the parameters are different from layer to layer.

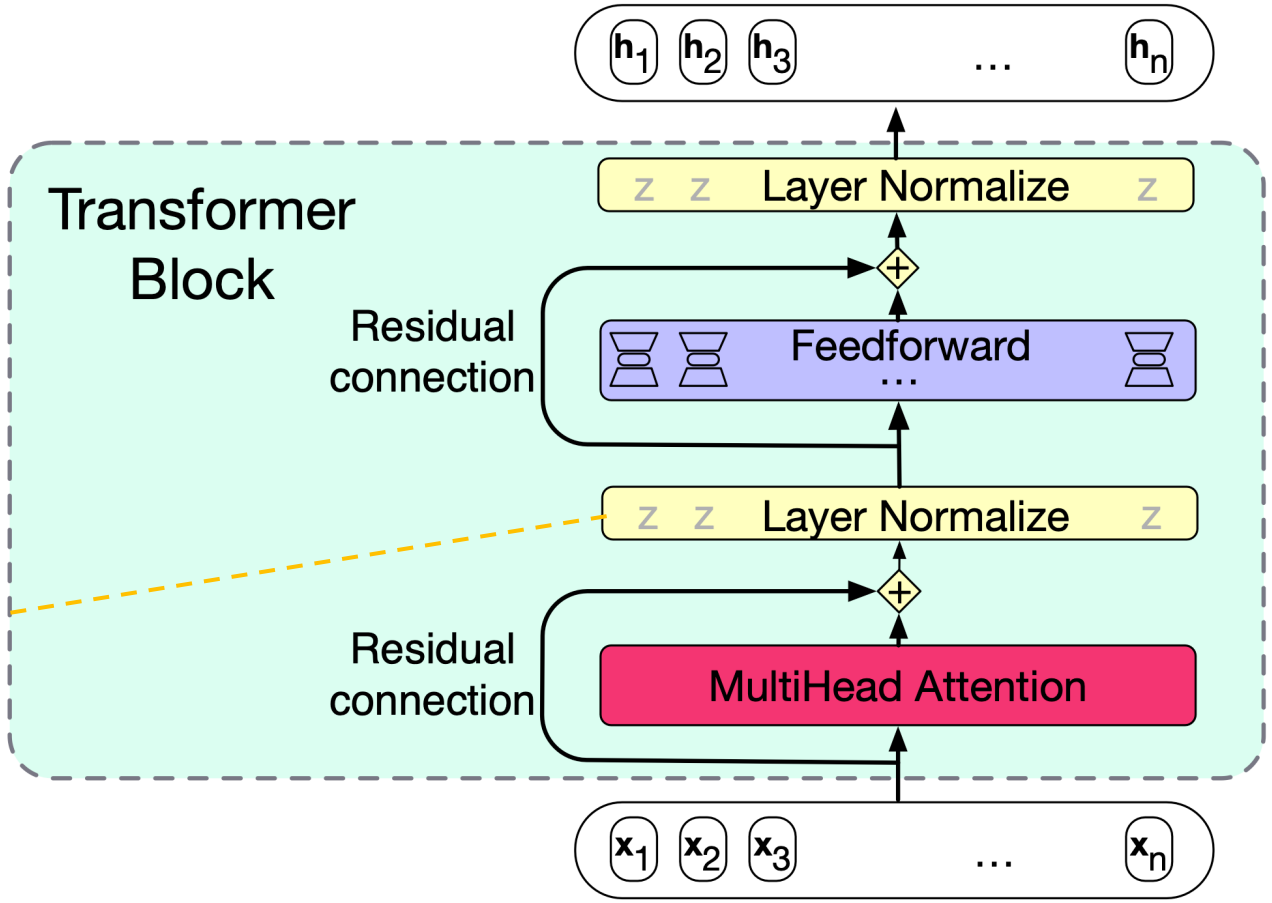
- **Layer Norm**

$$\mu = \frac{1}{d_h}\sum_{i=1}^{d_h} x_i$$

$$\sigma = \sqrt{\frac{1}{d_h}\sum_{i=1}^{d_h}(x_i-\mu)^2}$$

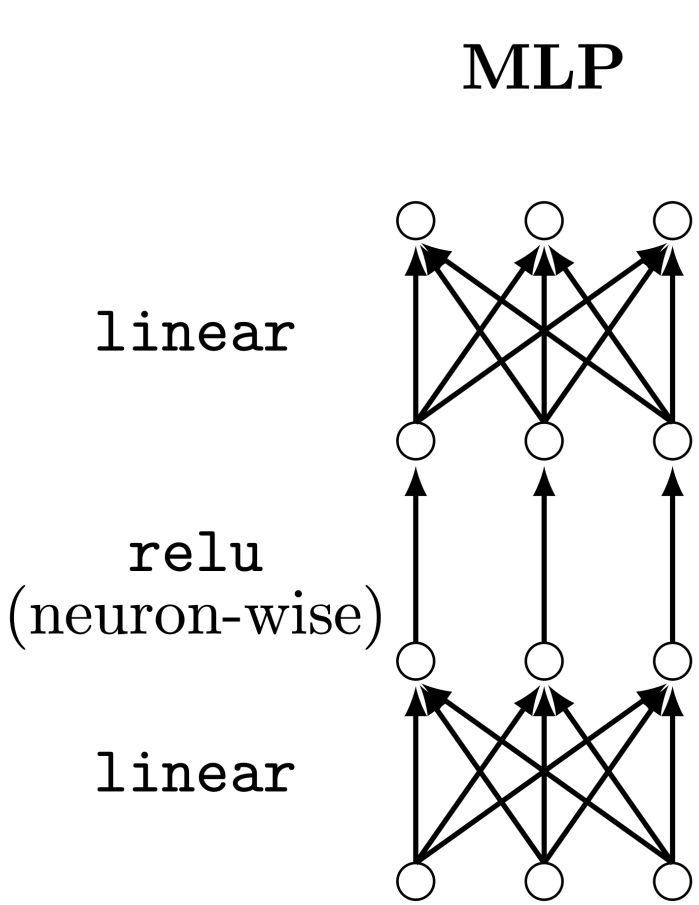
$$\hat{\mathbf{x}} = \frac{(\mathbf{x}-\mu)}{\sigma}$$

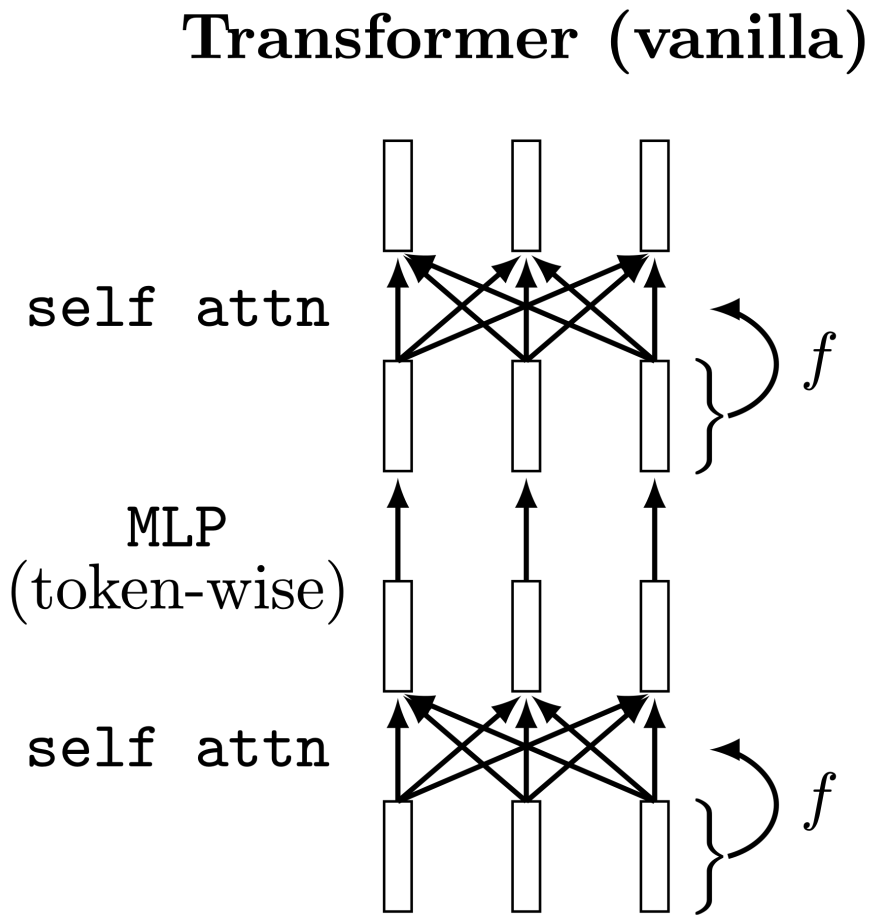
$$\text{LayerNorm} = \gamma\,\hat{\mathbf{x}} + \beta$$



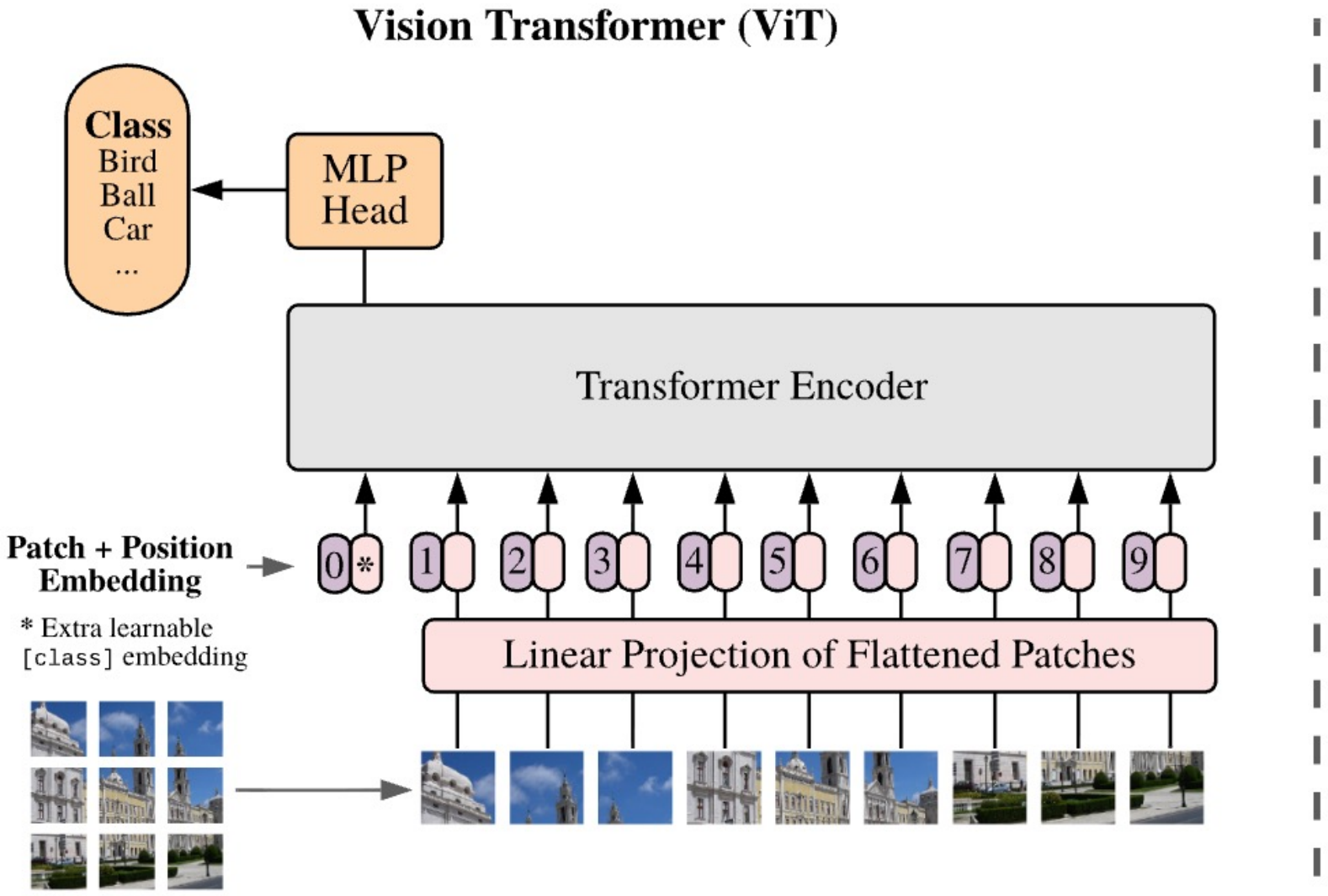

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{SelfAttention}(\mathbf{X}))$$
$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O}))$$

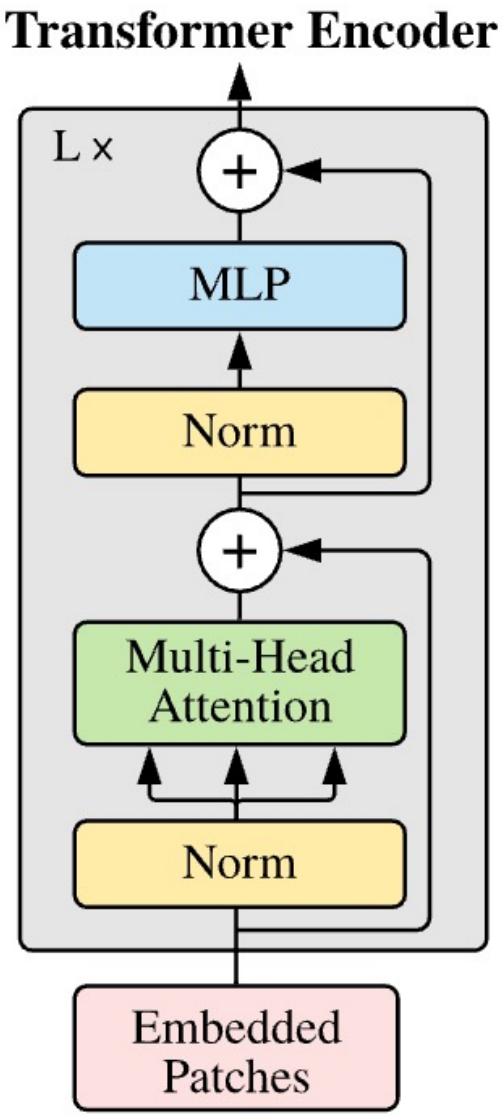
# 完整 Transformer 架构



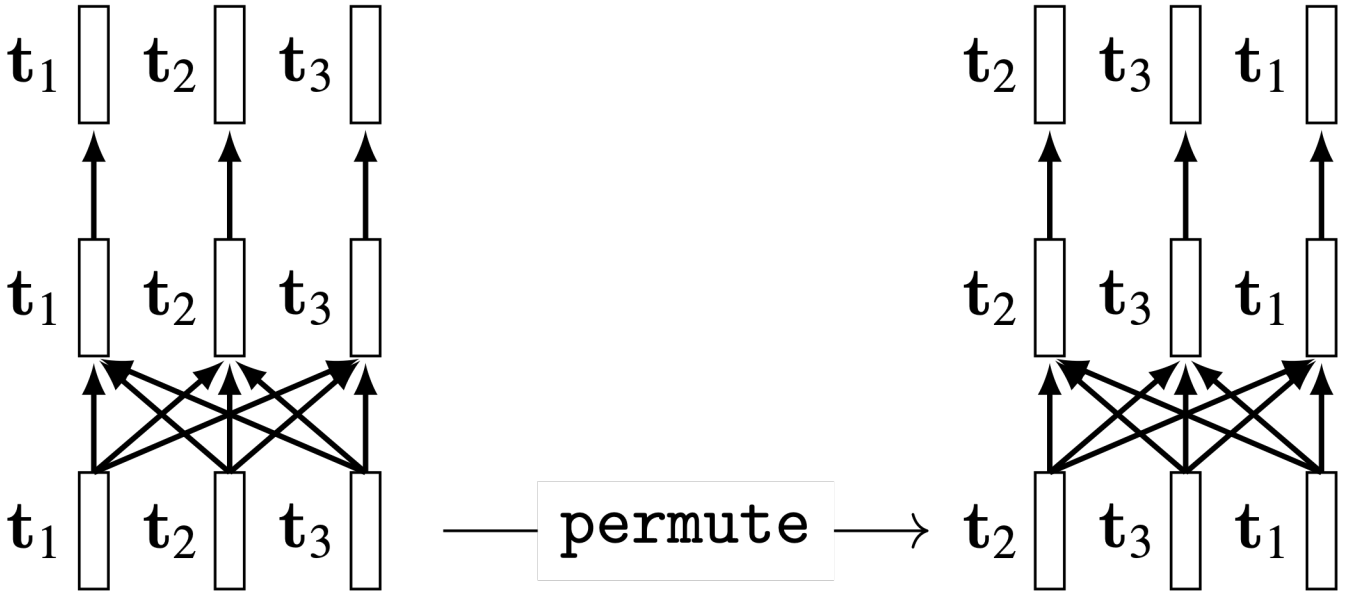



# ViT (Vision Transformer)



# 置换等变（Permutation Equivariance）




$$F_\theta(\texttt{permute}(\mathbf{T}_{\texttt{in}})) = \texttt{permute}(F_\theta(\mathbf{T}_{\texttt{in}}))$$

$$\texttt{attn}(\texttt{permute}(\mathbf{T}_{\texttt{in}})) = \texttt{permute}(\texttt{attn}(\mathbf{T}_{\texttt{in}}))$$

$$\downarrow$$

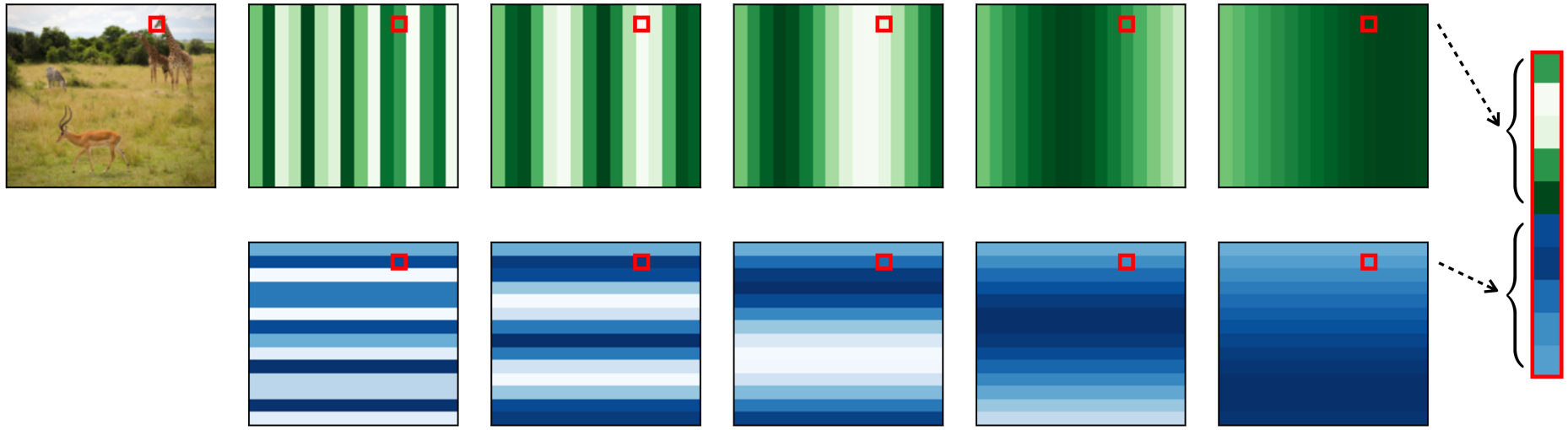
$$\texttt{transformer}(\texttt{permute}(\mathbf{T}_{\texttt{in}})) = \texttt{permute}(\texttt{transformer}(\mathbf{T}_{\texttt{in}}))$$

# 位置编码（Positional Encoding）

- 位置编码将一个表示位置的编码，拼接到每一个词元上
- 使用位置的周期性表示，即把坐标编码为一组正弦波在每个位置上取得的值所构成的向量：

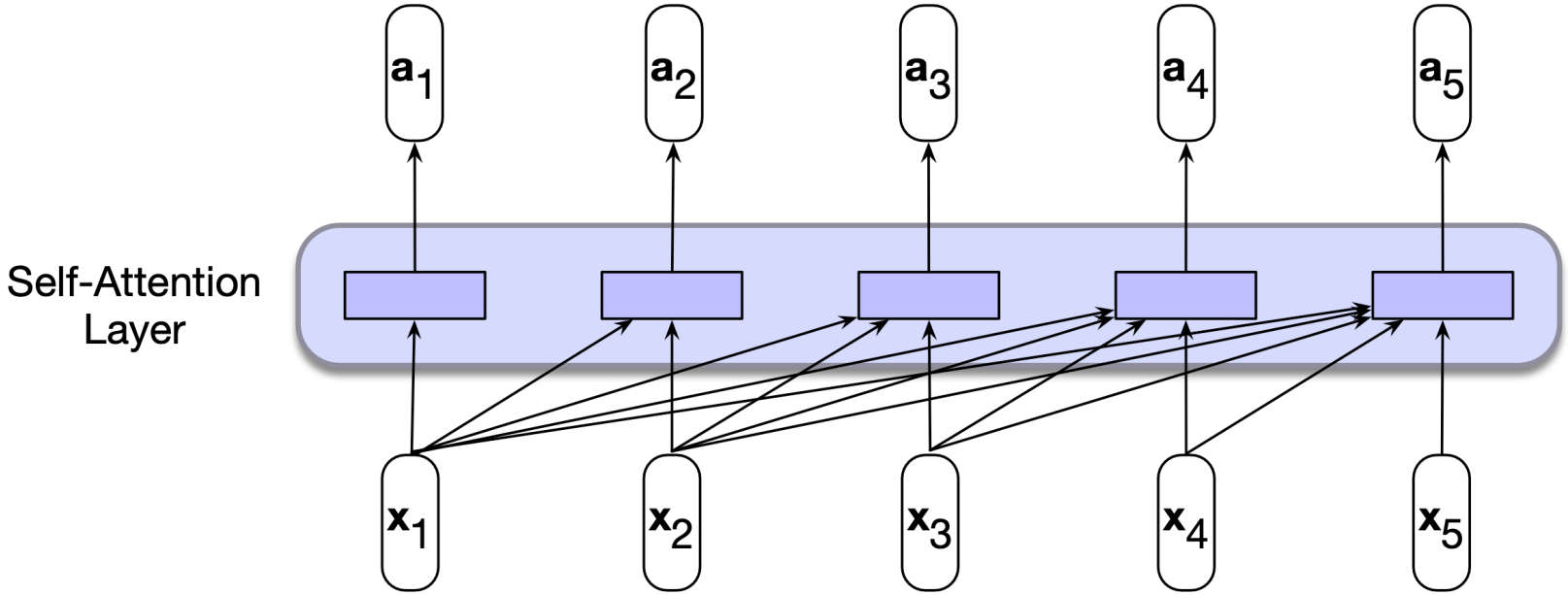
$$\mathbf{p}_x = [\sin(x), \sin(x/B), \sin(x/B^2), \ldots, \sin(x/B^P)]^\mathsf{T}$$
$$\mathbf{p}_y = [\sin(y), \sin(y/B), \sin(y/B^2), \ldots, \sin(y/B^P)]^\mathsf{T}$$
$$\mathbf{p} = \begin{bmatrix} \mathbf{p}_x \\ \mathbf{p}_y \end{bmatrix} \qquad B = 10000^{2/d}$$

- $\sin(x)$处的波长极短，频率极高。
- $\sin(x/B^P)$处波长极长、频率很低。



# Causal Self-Attention

- The model has access to all of the inputs up to and including the one under consideration, but no access to information about inputs beyond the current one.
  - Useful for creating **language model** and autoregressive generation
- The computation performed for each item is independent of all the other computations
  - Easily **parallelize** both forward inference and training

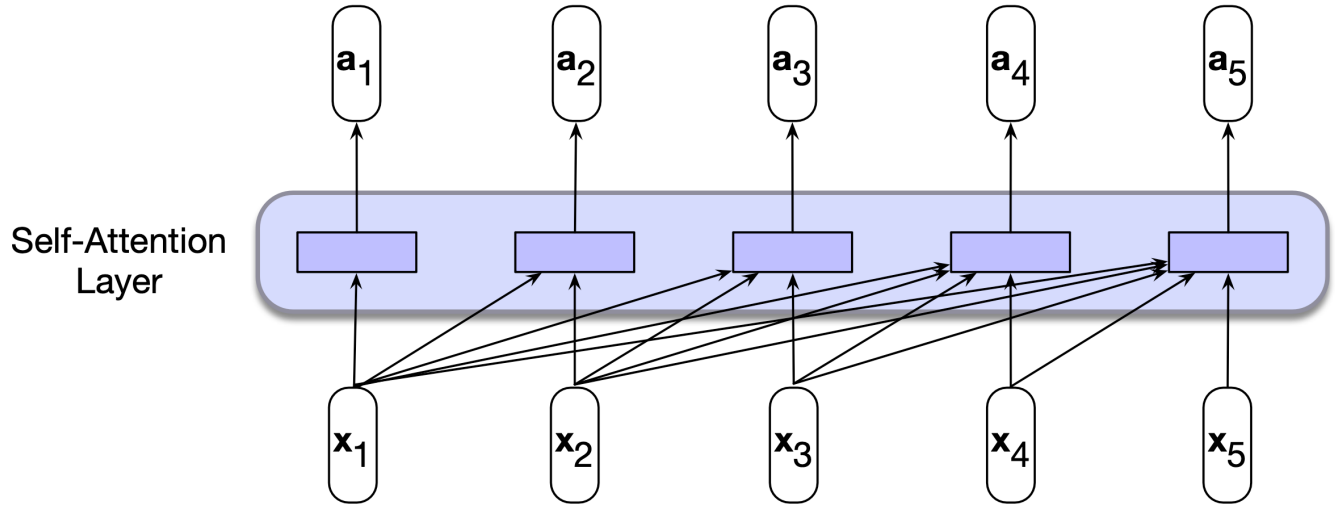


# Causal Self-Attention

- The calculation in $QK^T$ results in a score for each query value to every key value, *including those that follow the query*. This is inappropriate in the setting of language modeling: guessing the next word is pretty simple if you already know it!
- To fix this, the elements in the upper-triangular portion of the matrix are zeroed out (set to $-\infty$), thus eliminating any knowledge of words that follow in the sequence.

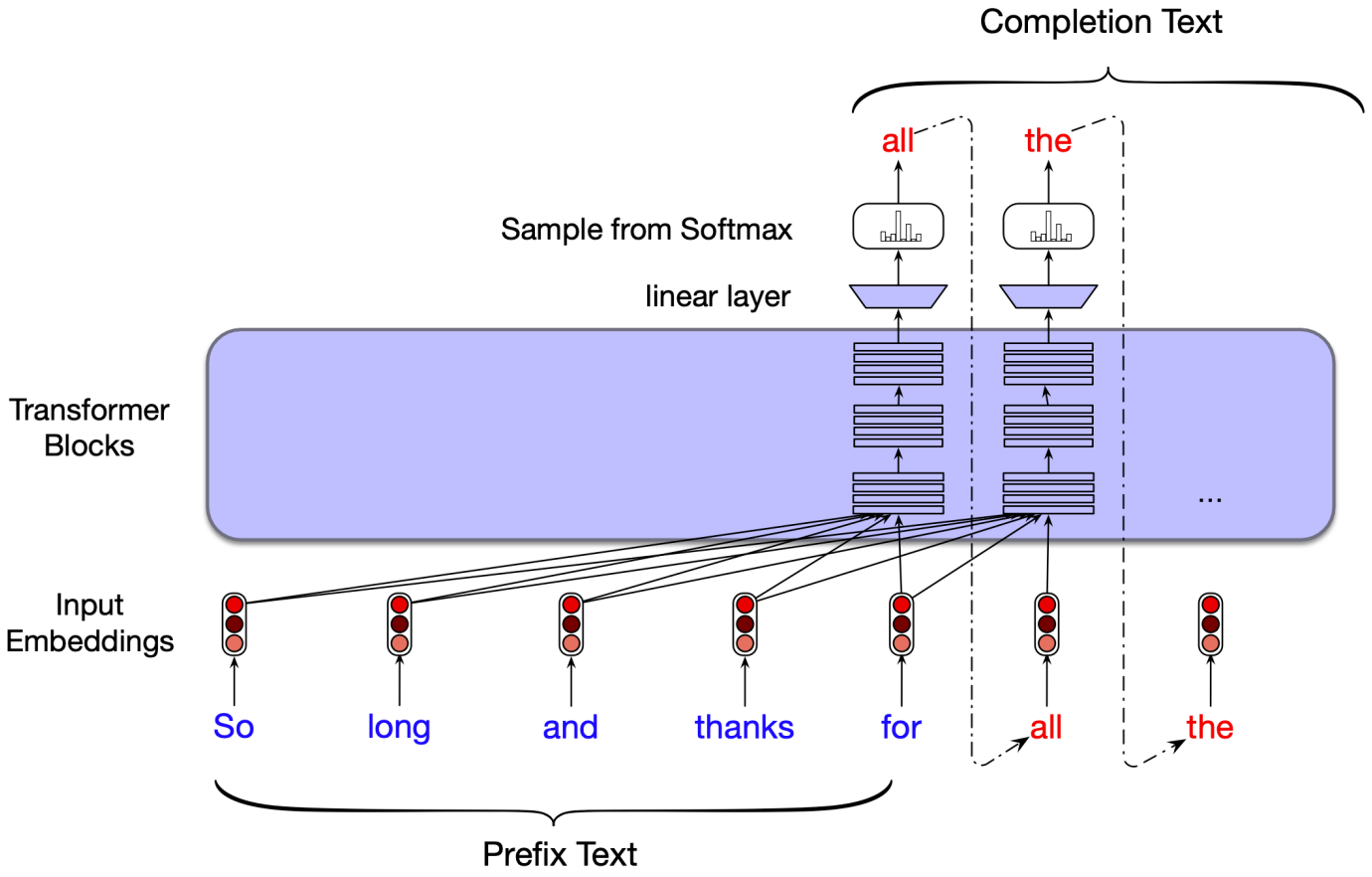
| | | | | | |
|---|---|---|---|---|---|
| N | q1•k1 | −∞ | −∞ | −∞ | −∞ |
| | q2•k1 | q2•k2 | −∞ | −∞ | −∞ |
| | q3•k1 | q3•k2 | q3•k3 | −∞ | −∞ |
| | q4•k1 | q4•k2 | q4•k3 | q4•k4 | −∞ |
| | q5•k1 | q5•k2 | q5•k3 | q5•k4 | q5•k5 |
| | | N | | | |

Masking out the future



# Large Language Models with Transformers

- **Conditional generation**: generating text conditioned on an input piece of text, a **prompt**.
- The fact that transformers have such long contexts (1024 or even 4096 tokens) makes them very powerful for conditional generation, because they can look back so far into the prompting text.




**Many practical NLP tasks can be cast as word prediction**

# Large Language Models with Transformers

- **Sentiment classification**

*"I like Jackie Chan"*



$$P(positive|The\ sentiment\ of\ the\ sentence\ "I\ like\ Jackie\ Chan"\ is:)$$

$$P(negative|The\ sentiment\ of\ the\ sentence\ "I\ like\ Jackie\ Chan"\ is:)$$

- Question Answering

```
Q: Who wrote the book ‘‘The Origin of Species"?  A:
```



$$P(w|Q:\ Who\ wrote\ the\ book\ "The\ Origin\ of\ Species"?\ A:)$$

$$P(w|Q:\ Who\ wrote\ the\ book\ "The\ Origin\ of\ Species"?\ A:\ Charles)$$

# Large Language Models with Transformers

## Text summarization

**Original Article**

The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For $89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says.

But not if you live in New England or surrounding states. "We will not ship snow to any states in the northeast!" says Waring's website, ShipSnowYo.com. "We're in the business of expunging snow!"

His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history. Most residents see the huge piles of snow choking their yards and sidewalks as a nuisance, but Waring saw an opportunity.

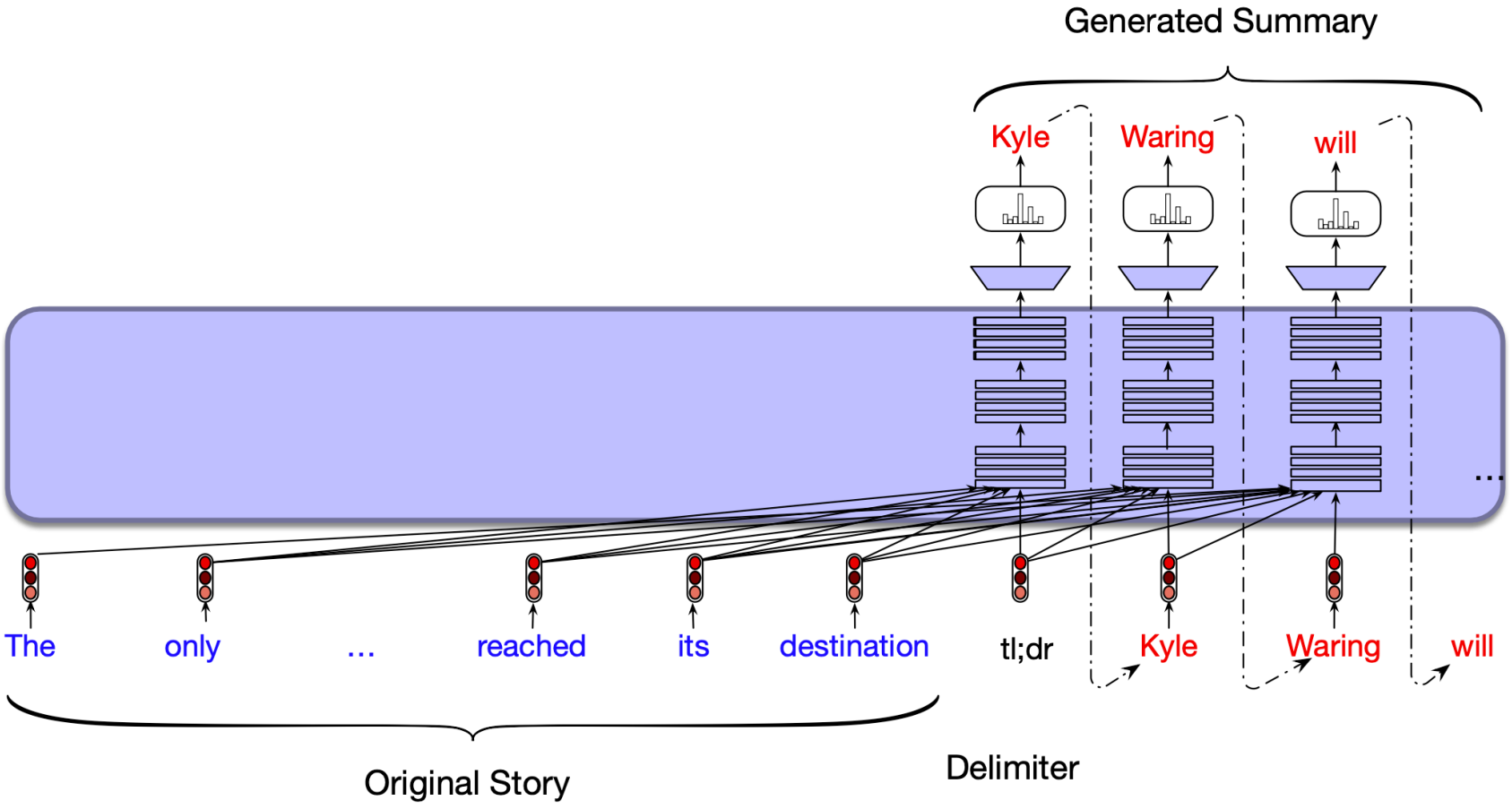
According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born. His business slogan: "Our nightmare is your dream!" At first, ShipSnowYo sold snow packed into empty 16.9-ounce water bottles for $19.99, but the snow usually melted before it reached its destination...

**Summary**

Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

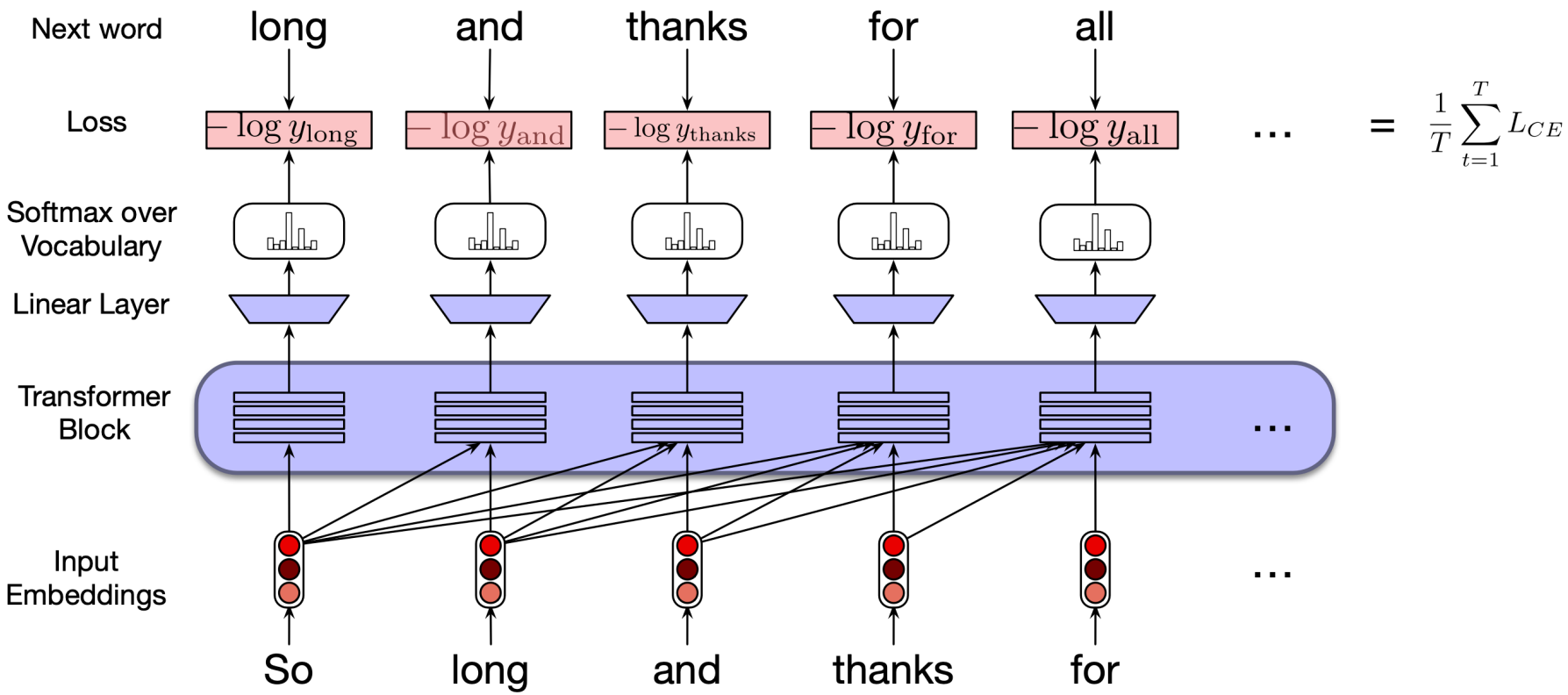
# Large Language Models with Transformers

- Cast summarization as language modeling by giving a large language model a text, and follow the text by a token like tl;dr;
  - **tl;dr** is short for something like '***too long; don't read***'



# LLM: Training Transformer

- Self-supervised training algorithm
- Each training item can be processed in parallel since the output for each element in the sequence is computed separately.



# CNN 与 Transformer

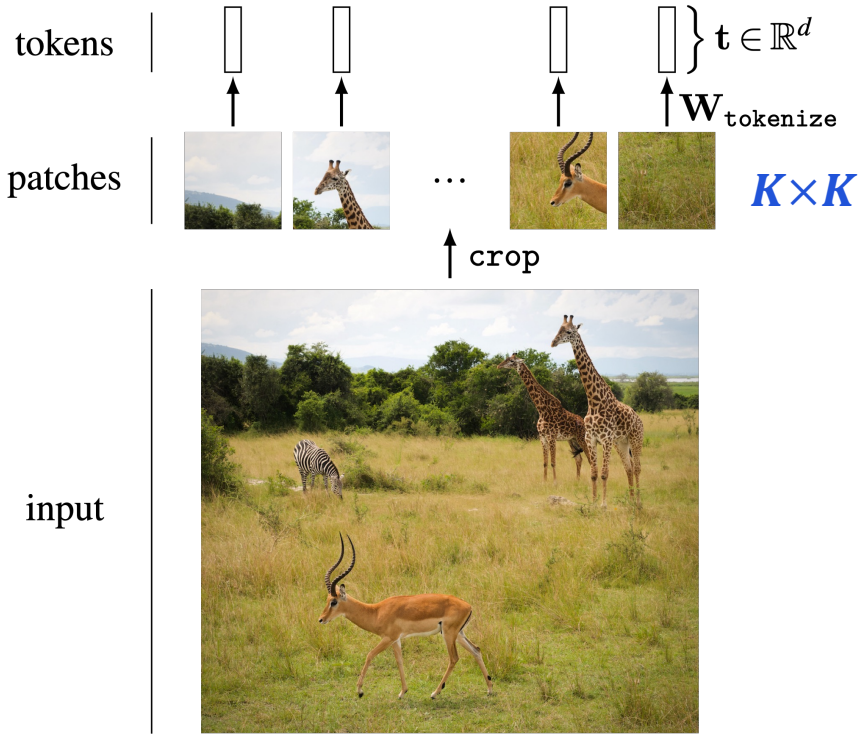
- CNN 与 Transformer 的相同点
  - Token 化
  - QKV 投影
  - 逐 Token 多层感知机
  - 其他：归一化、残差连接等

**核心思想：把信号处理问题分解为多个数据块**

- CNN 与 Transformer 的不同点
  - CNN 可具备非单一尺度的卷积核
  - Transformer 有注意力层

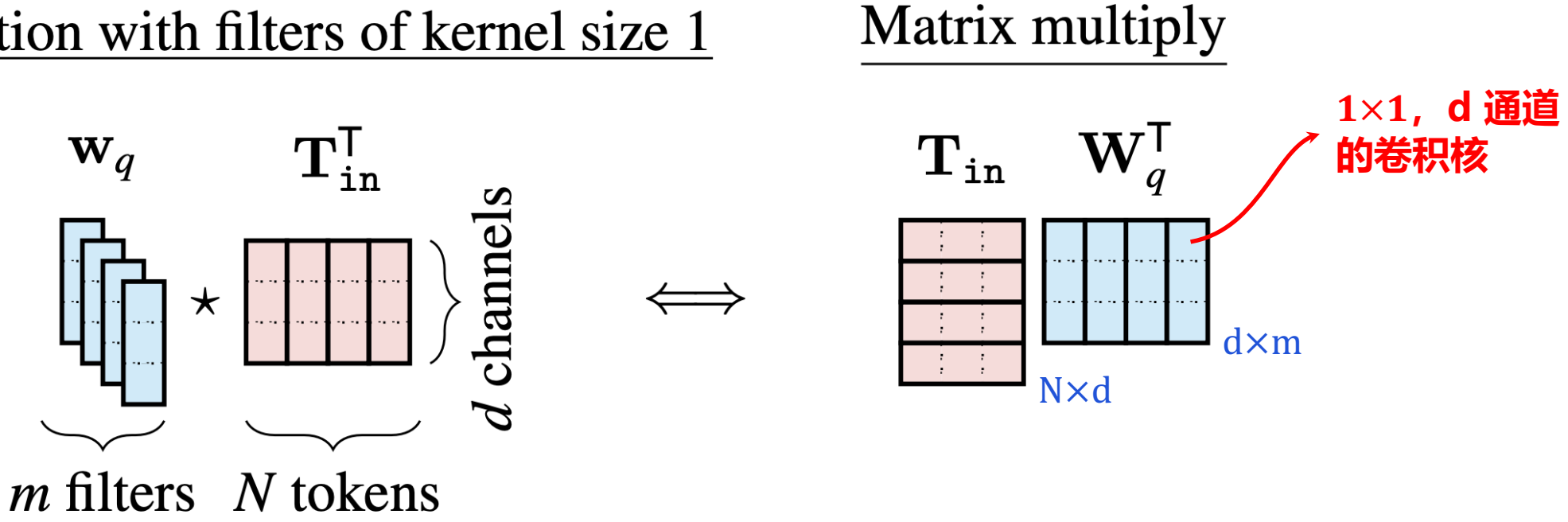
# Token 化

- Token 化操作可以写成卷积的形式
- Token 化可以写成一个**卷积核大小和步长都等于 K** 的卷积层




$$\mathbf{T}[n(N/K)+m, c_2] = b[c_2] + \sum_{c_1=1}^{C_{\text{in}}} \sum_{k_1,k_2=-K}^{K} w[c_1, c_2, k_1, k_2] x_{\text{in}}[c_1, Kn-k_1, Km-k_2]$$

# QKV 投影




- 线性代数中，矩阵相乘 AB，都可理解成多通道滤波器组 B与信号 A 的卷积运算
- 矩阵B 的每行代表一个 $1\times1$ 卷积核，A 中行对应索引，列对应通道

# 逐 Token 多层感知机

- Transformer 的MLP 中包含线性层和非线性层
- 线性层的本质上可表述为 $\mathbf{1}\times\mathbf{1}$ 卷积核的卷积运算

$$\mathbf{T}_{\text{out}} = \mathbf{T}_{\text{in}}\mathbf{W}^{\top}$$

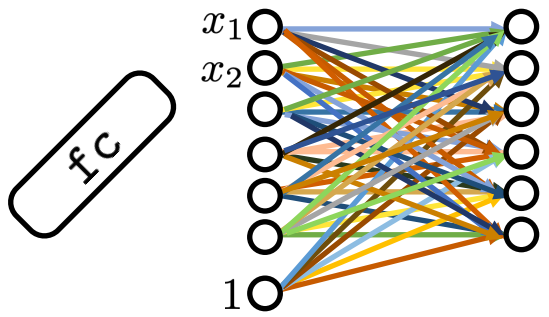
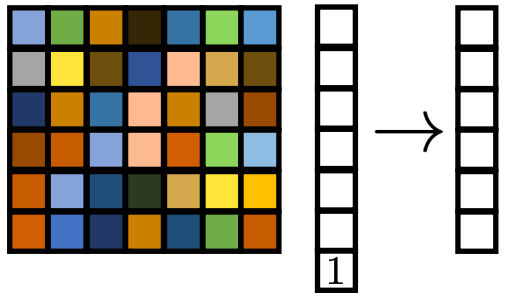
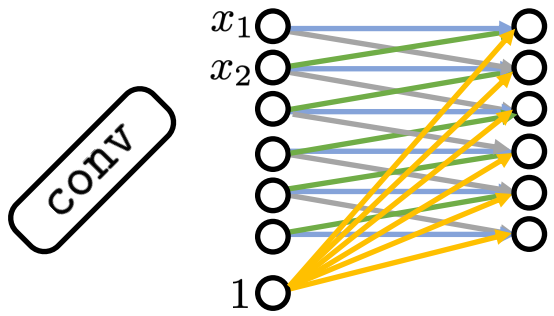
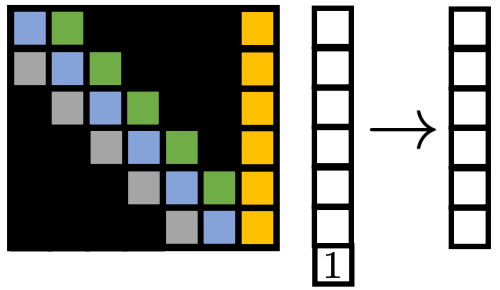
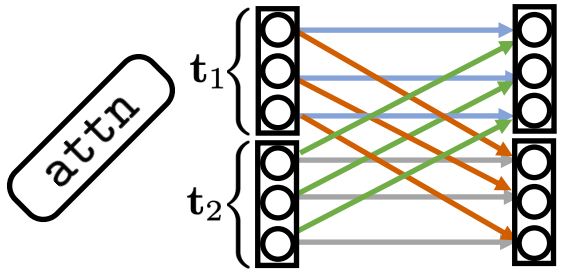
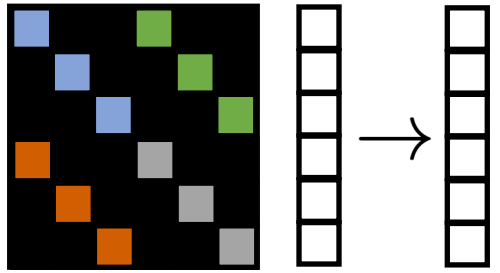
- 逐点非线性激活和 CNN 是一致的

# CNN 与 Transformer 的差异

- **Transformer：尺寸为 1的卷积 + 位置编码+注意力**
  - 若使用更大的卷积核，它将破坏 Transformer 的置换不变性
  - 为了解决空间结构信息问题，引入位置编码
  - 注意力层非卷积结构，不分解为独立的数据块，而是执行全局操作，解决了 CNN 中每个滤波器只能看到其感受野内的信息这一局限

- **CNN**：使用$K \times K$的滤波器，使相邻的图像区域被放在一起进行处理

# A Family Of Linear Layers

| Wiring graph | Matrix | Properties |
|---|---|---|
|  |  | Fixed input dimensionality<br>$N^2$ learnable parameters |
|  |  | Variable input dimensionality<br>$k+1$ learnable parameters ($k$ = kernel size)<br>$\texttt{conv}(\texttt{translate}(\mathbf{x})) = \texttt{translate}(\texttt{conv}(\mathbf{x}))$ |
|  |  | Variable input dimensionality<br>$\lvert\mathbf{W}_q\rvert + \lvert\mathbf{W}_k\rvert + \lvert\mathbf{W}_v\rvert$ learnable parameters<br>$\texttt{attn}(\texttt{permute}(\mathbf{T})) = \texttt{permute}(\texttt{attn}(\mathbf{T}))$ |

西南交通大学
Southwest Jiaotong University

# SCAI011112 视觉计算

# Thank You